
◆ Init Containers – Interview Q&A

1. What is an Init Container in Kubernetes?

Answer:

An Init Container is a special container that runs **before** the main application containers start. It is used to perform initialization tasks like setup, configuration, or dependency checks.

2. How does an Init Container differ from a regular container?

Answer:

- Init containers run **sequentially**
 - Each init container must **complete successfully**
 - Main containers start **only after** all init containers finish
 - Init containers run **once** per Pod lifecycle
-

3. Common use cases of Init Containers?

Answer:

- Waiting for a database or service to be available
 - Downloading configuration or secrets
 - Setting up permissions or directories
 - Running migration scripts
-

4. Can Init Containers share data with application containers?

Answer:

Yes, they can share data using **shared volumes** (e.g., `emptyDir`, `configMap`, `persistentVolume`).

5. What happens if an Init Container fails?

Answer:

- The Pod stays in **Init:CrashLoopBackOff**
 - Kubernetes retries the init container until it succeeds
 - Main containers never start until success
-

6. Can Init Containers have different images than app containers?

Answer:

Yes. Init containers often use **lightweight or utility images** (busybox, alpine, curl) separate from the app image.

7. Can Init Containers run in parallel?

Answer:

No. Init containers always run **one after another**, in the order defined in the Pod spec.

♦ Sidecar Containers – Interview Q&A

8. What is a Sidecar Container?

Answer:

A sidecar container is a helper container that runs **alongside** the main application container in the same Pod to provide supporting functionality.

9. How does a Sidecar Container differ from an Init Container?

Answer:

Init Container	Sidecar Container
Runs before app	Runs with app
Runs once	Runs continuously
Sequential	Parallel
Used for setup	Used for support

10. Common use cases of Sidecar Containers?

Answer:

- Log forwarding (Fluentd, Filebeat)
 - Service mesh proxies (Envoy)
 - Monitoring agents
 - Configuration reloaders
-

11. How do Sidecar Containers communicate with the main container?

Answer:

- Shared volumes
 - `localhost` networking
 - Shared process namespace (optional)
-

12. What happens if a Sidecar container crashes?

Answer:

- Pod behavior depends on the **restart policy**
 - Kubernetes restarts the failed container
 - Other containers continue running unless Pod fails
-

13. Can a Pod have multiple sidecar containers?

Answer:

Yes. A Pod can have **multiple sidecar containers**, each providing different functionality.

14. When would you prefer a Sidecar over an Init Container?

Answer:

Use a **Sidecar** when the task must:

- Run continuously
 - Monitor or assist the app at runtime
 - React to live traffic or logs
-

◆ Advanced / Scenario-Based Questions

15. How would you wait for a database before starting an app?

Answer:

Use an **Init Container** that:

- Repeatedly checks DB connectivity
 - Exits only when the DB is reachable
-

16. Can Sidecar containers be terminated independently?

Answer:

No. They share the Pod lifecycle. When the Pod is terminated, **all containers** stop together.

17. How does Kubernetes know when Init Containers are complete?

Answer:

Kubernetes checks the **exit code**. An exit code `0` means success.

18. Can Init Containers access Secrets and ConfigMaps?

Answer:

Yes. They can access Secrets and ConfigMaps the same way as app containers.

19. What are downsides of Sidecar containers?

Answer:

- Increased resource usage
 - More complex debugging
 - Tightly coupled lifecycle with the app
-

20. Real-world example using both?

Answer:

- **Init Container:** downloads config files
- **Sidecar Container:** continuously syncs logs to a central system